Metaheurísticas

Aula 1 — Otimização

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Conhecer o paradigma de otimização na engenharia;
2. Formalizar matematicamente problemas reais definindo variáveis, objetivos e restrições;
3. Distinguir problemas lineares de não-lineares, convexos de não-convexos, e contínuos de discretos;
4. Interpretar a paisagem de busca de um problema e identificar mínimos locais e globais; e
5. Compreender por que problemas reais resistem a métodos exatos.

1 Por que Otimizar?

Otimizar não é apenas resolver equações. É uma forma de pensar. Projetar um sistema, gerenciar um projeto, operar uma rede ou alocar recursos são variações da mesma pergunta: entre várias soluções possíveis, qual é a melhor dentro de certos limites?

Todo problema de otimização tem a mesma estrutura. Primeiro, existe um conjunto de soluções viáveis — às vezes pequeno, muitas vezes imenso. Depois, há um critério de desempenho: maximizar eficiência, lucro ou confiabilidade; minimizar perdas, custos ou riscos. Por fim, há restrições — físicas, operacionais ou econômicas — que delimitam o que pode ou não pode ser feito.

Métodos clássicos como o cálculo variacional e a programação linear resolvem muitos desses problemas. Eles funcionam bem quando as funções são contínuas, diferenciáveis e, de preferência, lineares. Mas sistemas reais são não-lineares, muitas decisões envolvem variáveis inteiras ou binárias, e o número de combinações possíveis pode crescer a ponto de tornar inviável examinar cada alternativa.

Nesse contexto, surgem as metaheurísticas. Elas não prometem a solução perfeita — e, na prática, quase nunca precisamos dela. O que oferecem é algo mais útil: boas soluções obtidas em tempo razoável. Em problemas grandes e complexos, isso faz toda a diferença.

2 Formulação Matemática

Independentemente da aplicação, um problema de otimização pode ser descrito por três componentes fundamentais:

1. **Variáveis de decisão (x):** O que podemos controlar.
2. **Função Objetivo ($f(x)$):** O critério de qualidade da decisão.
3. **Restrições ($\mathcal{C}$):** As regras que limitam nossas escolhas.

Definição — Problema de Otimização Geral
Seja $\boldsymbol{x} = [x_1, x_2, ..., x_n]^T$ um vetor de variáveis de decisão no espaço $\mathbb{R}^n$. O problema padrão é formulado como:

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & f(\boldsymbol{x}) \\ \text{sujeito a:} \quad & g_{i(\boldsymbol{x})} = 0, \quad i = 1, ..., p \\ & h_{j(\boldsymbol{x})} \leq 0, \quad j = 1, ..., m \end{aligned}$$

Onde:
- $f : \mathbb{R}^n \to \mathbb{R}$ é a função objetivo.
- $g_{i(\boldsymbol{x})}$ são as **restrições de igualdade** (leis de Kirchhoff, balanço de potência).
- $h_{j(\boldsymbol{x})}$ são as **restrições de desigualdade** (limites físicos, térmicos e de segurança).

A região definida pelas restrições é chamada de **Espaço Viável** (Ω). Se buscamos maximizar $f(x)$, basta minimizar $-f(x)$.

2.1 Componentes do problema na EE

Variáveis de decisão. São as grandezas que o projetista pode controlar: tensão em barras de geração (V_g), potência ativa despachada (P_g), posição de tap de transformadores (t_k), estado de chaves ($u_k \in \{0, 1\}$), número de capacitores ($q_i \in \mathbb{Z}_+$).

Função objetivo. Quantifica o critério de qualidade. A escolha é uma decisão de modelagem: otimizar a função errada produz soluções tecnicamente ótimas mas operacionalmente inadequadas. Funções comuns em EE:

Custo de geração
$$f = \sum_i \left(a_i P_{g,i}^2 + b_i P_{g,i} + c_i\right)$$

Perdas na rede
$$f = \sum_{k \in \mathcal{L}} R_k \, |I_k|^2$$

Desvio de tensão
$$f = \sum_i \left(V_i - V_i^{\text{ref}}\right)^2$$

Emissões de CO_2
$$f = \sum_i e_i P_{g,i}$$

Restrições. Expressam leis físicas e limites operacionais:
- **Igualdade** ($h(\boldsymbol{x}) = 0$): balanço de potência ativa e reativa (equações de fluxo de potência).
- **Desigualdade** ($g(\boldsymbol{x}) \leq 0$): limites de geração ($P_{g,i}^{\min} \leq P_{g,i} \leq P_{g,i}^{\max}$), limites de tensão ($V^{\min} \leq V_i \leq V^{\max}$), capacidade de corrente ($|I_k| \leq I_k^{\max}$).

3 Classificação de Problemas de Otimização

A natureza do problema determina quais métodos são aplicáveis. Classificamos os problemas segundo quatro eixos principais.

3.1 Linearidade vs. Não-Linearidade

Essa distinção determina se podemos usar a álgebra linear e o método Simplex, ou se precisaremos de métodos iterativos não-lineares.

Definição — Problema Linear (LP)
Um problema é linear se, e somente se, a função objetivo $f(\boldsymbol{x})$ e **todas** as restrições $g(\boldsymbol{x}), h(\boldsymbol{x})$ são funções lineares das variáveis de decisão.

Matematicamente, satisfazem o princípio da superposição: $f(\alpha x + \beta y) = \alpha f(x) + \beta f(y)$.

Exemplo
Linear: Despacho Econômico DC. As equações de fluxo de potência são linearizadas assumindo tensão constante e pequenos ângulos ($\sin\theta \approx \theta$).

Não-Linear: Fluxo de Potência AC. A relação entre potência, tensão e ângulo envolve produtos e funções trigonométricas ($P = V_1 V_2 \sin(\theta)$), tornando o problema inerentemente não-linear (NLP).

3.2 Convexidade

Talvez o conceito mais importante para a aplicação de metaheurísticas. A convexidade determina se "descer a montanha" leva inevitavelmente ao ponto mais baixo do terreno — ou se existem armadilhas no caminho. Ela tem **duas faces independentes**: a forma da função objetivo e a forma da região viável. Ambas precisam ser convexas para que o problema como um todo seja convexo.

Definição — Conjunto Convexo
Um conjunto $\Omega \subseteq \mathbb{R}^n$ é **convexo** se o segmento de reta entre quaisquer dois pontos de Ω está inteiramente contido em Ω: para todo $\boldsymbol{x}, \boldsymbol{y} \in \Omega$ e todo $\lambda \in [0, 1]$,

$$\lambda \boldsymbol{x} + (1 - \lambda)\boldsymbol{y} \in \Omega.$$

Geometricamente: não há "reentrâncias" ou "buracos" — o conjunto parece uma tigela vista de cima, nunca uma meia-lua ou um arquipélago.

Definição — Função Convexa
Uma função $f : \mathbb{R}^n \to \mathbb{R}$ é **convexa** se, para quaisquer dois pontos distintos $\boldsymbol{x}, \boldsymbol{y} \in \mathrm{dom}(f)$ e para todo $\lambda \in [0, 1]$:

$$f(\lambda \boldsymbol{x} + (1 - \lambda)\boldsymbol{y}) \leq \lambda f(\boldsymbol{x}) + (1 - \lambda) f(\boldsymbol{y}).$$

Geometricamente: **o segmento de reta que une dois pontos quaisquer do gráfico de f situa-se acima ou sobre o gráfico entre esses dois pontos.** A função é **estritamente convexa** quando a desigualdade é estrita para $\lambda \in (0, 1)$.

Consequência fundamental: **uma função convexa possui apenas um mínimo local, que coincide com o mínimo global** — desde que o domínio também seja convexo.

Definição — Problema de Otimização Convexo
Um problema de minimização é **convexo** se, e somente se, **simultaneamente**:

1. A função objetivo f é convexa.
2. O conjunto viável Ω é convexo — o que requer que as restrições de desigualdade $h_{j(\boldsymbol{x})} \leq 0$ sejam funções convexas de $\boldsymbol{x}$, e que as restrições de igualdade $g_{i(\boldsymbol{x})} = 0$ sejam **afins** (lineares).

Quando ambas as condições são satisfeitas, qualquer mínimo local é necessariamente global. Violar **qualquer uma** das duas é suficiente para que mínimos locais espúrios possam existir.

⚠ Atenção

As duas condições são independentes e igualmente necessárias. Uma função convexa sobre um conjunto não-convexo **não** define um problema convexo — e vice-versa. O erro mais comum em engenharia é concentrar a análise na função objetivo e ignorar a geometria de Ω.

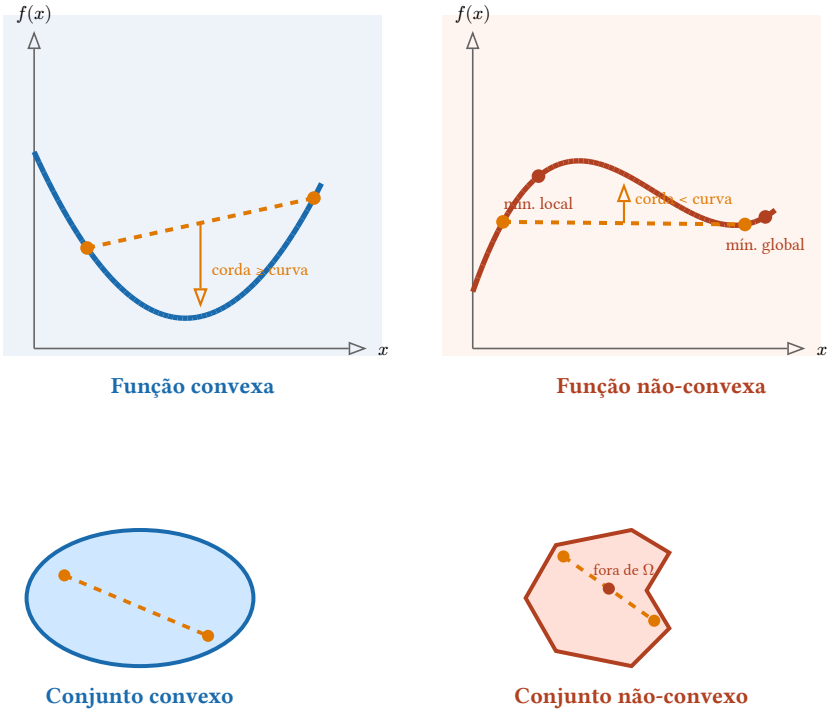

Figura 1: **Linha superior — função:** à esquerda, função convexa (corda acima da curva, único mínimo global); à direita, função não-convexa (corda abaixo da curva num trecho, mínimos locais espúrios). **Linha inferior — conjunto viável:** à esquerda, conjunto convexo (qualquer segmento entre dois pontos permanece dentro de Ω); à direita, conjunto não-convexo (o segmento tracejado atravessa uma região exterior a Ω, violando a definição). Um problema de otimização é convexo somente quando **ambas** as linhas mostram o caso favorável simultaneamente.

A ausência de convexidade — seja na função, seja no conjunto viável — cria **múltiplos mínimos locais**. Métodos baseados em gradiente convergem para o primeiro encontrado e param, sem qualquer garantia de que existe um vale mais profundo adiante.

Exemplo

Convexo (função e conjunto): minimização de perdas em rede de distribuição radial com cargas de corrente constante. A função objetivo é quadrática convexa nas correntes, e o conjunto viável é definido por restrições lineares — um poliedro convexo. Qualquer descida leva ao ótimo global.

Não-convexo pela função: despacho econômico com **efeito válvula**. A curva de custo oscila quando válvulas adicionais se abrem, criando múltiplos vales locais mesmo que o conjunto viável (limites de geração) seja um intervalo convexo.

Não-convexo pelo conjunto: planejamento de expansão de transmissão. Decisões de investimento são binárias ($x_i \in \{0, 1\}$), tornando Ω um conjunto discreto — geometricamente, uma

nuvem de pontos isolados. Mesmo que a função objetivo fosse convexa, a descontinuidade do domínio destrói a convexidade do problema. A restrição de igualdade das equações de fluxo de potência (não-linear) agrava o problema: ela define uma variedade não-convexa em $\mathbb{R}^n$, e qualquer ponto nessa variedade é potencialmente isolado de seus "vizinhos naturais".

Não-convexo em ambos: OPF com variáveis mistas (tensão contínua + estado de chaves binário). Esse é o caso genérico em EE — e a principal motivação para metaheurísticas.

3.3 Natureza das Variáveis

Muitas decisões em engenharia são binárias (liga/desliga) ou inteiras (número de transformadores), não contínuas.

- **Contínuas:** $x \in \mathbb{R}$. Ex: tensão, potência, corrente.
- **Discretas/Inteiras:** $x \in \mathbb{Z}$. Ex: posição do tap do transformador, banco de capacitores.
- **Mistas (MINLP):** problemas que combinam ambas. São notavelmente difíceis de resolver, pois os métodos de relaxação contínua e de enumeração combinatória precisam ser integrados.

3.4 Estático vs. Dinâmico

Definição — Otimização Dinâmica
Ocorre quando a decisão tomada no instante t afeta as restrições ou custos no instante $t+1$. O problema não pode ser resolvido como uma sequência de "fotos" independentes; ele deve ser resolvido considerando todo o horizonte de tempo simultaneamente.

Problema Estático (Snapshot)
O estado futuro não depende do atual.

Exemplo: **Fluxo de Potência Ótimo (OPF)** para uma hora específica. O objetivo é ajustar a geração para aquela carga instantânea.

Problema Dinâmico (Temporal)
Existe acoplamento temporal (efeito memória).

Exemplo: **Unit Commitment** ou **Baterias**. A energia armazenada na bateria às 14h depende do quanto foi carregado às 13h. Não se pode otimizar as 14h sem saber o estado das 13h.

3.5 Síntese da Classificação

A Tabela 1 resume os critérios discutidos e exemplifica cada categoria no contexto de EE.

Critério	Categorias	Exemplo em EE
Linearidade	Linear (LP) / Não-linear (NLP)	Custo linear vs. equações de fluxo AC
Convexidade	Convexo / Não-convexo	Despacho sem efeito válvula vs. com
Natureza das variáveis	Contínua / Inteira / Mista (MINLP)	Tensão contínua; nº de capacitores inteiro
Nº de objetivos	Mono-objetivo / Multi-objetivo	Mínimo custo vs. mínimo custo e mínima emissão
Dinâmica	Estático / Dinâmico	OPF snapshot vs. despacho horário

Tabela 1: Classificação de problemas de otimização em engenharia elétrica.

⚠️ **Atenção**
A maioria dos problemas reais de engenharia elétrica são **MINLP não-convexos**, frequentemente com múltiplos objetivos conflitantes. Essa combinação inviabiliza soluções exatas para instâncias de grande porte e justifica o uso de metaheurísticas.

4 Por que Métodos Exatos Falham?

Métodos exatos (*branch-and-bound*, programação dinâmica, pontos interiores) garantem a solução ótima global, mas a um custo que cresce combinatorialmente com o tamanho do problema.

Definição — NP-difícil
Um problema de otimização é **NP-difícil** se existe uma redução em tempo polinomial de qualquer problema em NP para ele — isto é, ele é ao menos tão difícil quanto qualquer problema na classe NP. É importante notar que isso **não** implica, por si só, que nenhum algoritmo polinomial existe: essa afirmação equivaleria a provar $P \neq NP$, questão em aberto. Na prática, NP-difícil significa que **não se conhece** algoritmo com tempo de execução polinomial no pior caso, e a comunidade científica acredita fortemente que tal algoritmo não existe. O espaço de soluções cresce combinatorialmente, tornando a enumeração exaustiva inviável mesmo para instâncias moderadas.

Exemplos em EE: configuração ótima de redes de distribuição, planejamento de expansão de transmissão, programação de unidades geradoras (*Unit Commitment*). Para esses problemas, metaheurísticas oferecem o melhor compromisso prático entre qualidade da solução e tempo computacional.

5 A Paisagem de Busca

Com a taxonomia estabelecida, podemos agora desenvolver uma linguagem geométrica para descrever a dificuldade de um problema. O conceito de **paisagem de busca** transforma a função objetivo num objeto que podemos analisar e — em baixa dimensão — visualizar.

Definição — Paisagem de Busca
Dado o espaço viável $\mathcal{F}$ e a função objetivo f, a **paisagem de busca** é o par $(\mathcal{F}, f)$ interpretado geometricamente: cada ponto $\boldsymbol{x} \in \mathcal{F}$ tem "altitude" $f(\boldsymbol{x})$. Estruturas relevantes:
- **Mínimo local**: $\boldsymbol{x}^l$ tal que $f(\boldsymbol{x}^l) \leq f(\boldsymbol{x})$ para todo $\boldsymbol{x}$ na vizinhança $\mathcal{N}(\boldsymbol{x}^l)$.
- **Mínimo global**: $\boldsymbol{x}^*$ tal que $f(\boldsymbol{x}^*) \leq f(\boldsymbol{x})$ para todo $\boldsymbol{x} \in \mathcal{F}$.
- **Bacia de atração**: conjunto de pontos de onde algoritmos de descida convergem para o mesmo mínimo local.

A estrutura da paisagem determina diretamente a dificuldade do problema para qualquer algoritmo de busca:

- **Paisagem unimodal** (um único vale): qualquer método de descida encontra o ótimo global a partir de qualquer ponto de partida. Problemas convexos têm sempre essa forma.
- **Paisagem multimodal** (múltiplos vales): algoritmos de descida ficam presos no primeiro mínimo local encontrado. A maioria dos problemas de EE tem essa estrutura quando modelos realistas são adotados.
- **Paisagem rugosa**: pequenas perturbações em $\boldsymbol{x}$ causam variações abruptas em f. Dificulta tanto a orientação por gradiente quanto a vizinhança local.
- **Paisagem plana** (platôs): regiões onde $\nabla f \approx 0$; a informação de gradiente não orienta a busca.

5.1 Medindo a Paisagem: Análise de Paisagem de Busca (FLA)

Descrever a paisagem como "rugosa" ou "multimodal" é útil para construir intuição, mas insuficiente para **comparar** problemas ou justificar a escolha de um algoritmo. A **Fitness Landscape Analysis** (FLA) oferece métricas quantitativas que transformam propriedades qualitativas em números — computáveis mesmo quando o ótimo global é desconhecido.

Todas as métricas abaixo são definidas sobre amostras do espaço de busca e, portanto, não requerem enumeração completa. Isso as torna aplicáveis a problemas reais de grande porte.

5.1.1 Correlação Distância–Fitness (FDC)

Proposta por Jones e Forrest (1995), a FDC mede se existe uma tendência global de "o ótimo está onde a função é menor": soluções mais próximas do ótimo global $\boldsymbol{x}^*$ tendem a ter valores menores?

Seja $\boldsymbol{x}^*$ o ótimo global (ou a melhor solução conhecida). Para uma amostra $\left\{\boldsymbol{x}^{\{(k)\}}\right\}_{\{k=1\}}^{N}$, defina:

$$d_k = d\left(\boldsymbol{x}^{\{(k)\}}, \boldsymbol{x}^*\right), \quad f_k = f\left(\boldsymbol{x}^{\{(k)\}}\right).$$

A FDC é o coeficiente de correlação de Pearson entre $\boldsymbol{d}$ e $\boldsymbol{f}$:

$$\rho_{\text{FDC}} = \frac{\sum_k (d_k - |(d))(f_k - |(f))}{\sqrt{\sum_k (d_k - |(d))^2}\sqrt{\sum_k (f_k - |(f))^2}}.$$

Interpretação prática:

ρ_{FDC}	Classificação	Implicação
$\rho > 0.15$	Fácil	Descida global orienta para $\boldsymbol{x}^*$; HC pode funcionar bem
$\lvert\rho\rvert \leq 0.15$	Difícil	Sem correlação útil; busca local é basicamente aleatória
$\rho < -0.15$	Enganosa	Descida afasta do ótimo; algoritmos gulosos pioram a solução

Tabela 2: Classificação de paisagens pela FDC segundo Jones e Forrest (1995).

Limitação: a FDC requer conhecer $\boldsymbol{x}^*$ (ou uma boa aproximação). Em problemas onde o ótimo é completamente desconhecido, usa-se a melhor solução encontrada por qualquer método como proxy — o que introduz viés, mas ainda fornece informação útil para comparação relativa entre instâncias.

5.1.2 Autocorrelação e Comprimento de Correlação

A rugosidade de uma paisagem pode ser medida ao longo de uma **caminhada aleatória**: partindo de $\boldsymbol{x}^{\{(0)\}}$, gera-se a sequência $\boldsymbol{x}^{\{(0)\}}, \boldsymbol{x}^{\{(1)\}}, \ldots$ aplicando perturbações unitárias aleatórias. A autocorrelação da série $f^{\{(t)\}} = f\big(\boldsymbol{x}^{\{(t)\}}\big)$ no lag s é:

$$r(s) = \frac{\sum_{\{t=1\}}^{\{N-s\}} \big(f^{\{(t)\}} - |(f)\big)\big(f^{\{(t+s)\}} - |(f)\big)}{\sum_{\{t=1\}}^{N} \big(f^{\{(t)\}} - |(f)\big)^2}.$$

O **comprimento de correlação** ξ é definido como o menor s tal que $r(s)$ cai abaixo de $\frac{1}{e} \approx 0.37$:

$$\xi = \min\big\{s \in \mathbb{Z}_+ : r(s) < e^{\{-1\}}\big\}.$$

Interpretação: ξ mede por quantos passos a informação de f se propaga na vizinhança. Paisagens rugosas têm ξ pequeno (1–2 passos); paisagens suaves têm ξ grande. Em termos práticos:

- ξ **pequeno** $\longrightarrow$ operadores de busca local têm "visão curta"; reinícios frequentes são necessários; operadores de mutação forte (ampla perturbação) são mais adequados.
- ξ **grande** $\longrightarrow$ gradiente local é informativo por muitos passos; busca local intensa (Hill Climbing, SA com temperatura baixa) funciona bem; crossover de GA preserva blocos de construção úteis.

5.1.3 Razão de Ótimos Locais por Amostragem de Bacias

O número absoluto de ótimos locais é intratável para estimar diretamente. Uma alternativa prática é executar R reinícios de Hill Climbing (ou descida de gradiente) a partir de pontos iniciais uniformemente amostrados e contar quantos **atratores distintos** são encontrados:

$$\widehat{M} = |\{\boldsymbol{x}_r^* : r = 1, \ldots, R, \quad \boldsymbol{x}_r^* \text{ é ótimo local}\}|.$$

A **razão de ótimos locais por amostra** $\frac{\widehat{M}}{R}$ estima indiretamente a densidade de atratores. Se $R = 200$ reinícios convergem para 47 pontos distintos, a paisagem é altamente multimodal — e algoritmos que mantêm população diversa (GA, DE) serão mais robustos do que buscas de trajetória única.

5.1.4 Neutralidade

Em espaços discretos, é comum que muitos vizinhos de $\boldsymbol{x}$ tenham exatamente o mesmo valor $f(\boldsymbol{x}') = f(\boldsymbol{x})$. A **fração de neutralidade** é:

$$\nu(\boldsymbol{x}) = \frac{|\{\boldsymbol{x}' \in \mathcal{N}(\boldsymbol{x}) : f(\boldsymbol{x}') = f(\boldsymbol{x})\}|}{|\mathcal{N}(\boldsymbol{x})|}.$$

Alta neutralidade (ν próximo de 1) indica platôs extensos onde o gradiente não orienta a busca. Isso é particularmente relevante em problemas de EE com variáveis inteiras: a função objetivo pode ser completamente insensível a pequenas variações discretas (ex: mudar o tap de um transformador por uma posição não altera o custo de forma detectável em muitas configurações de rede).

5.1.5 Síntese: Diagnóstico Antes de Escolher o Algoritmo

Métrica	O que mede	Algoritmo favorecido
FDC alto (> 0.15)	Orientação global confiável	Hill Climbing, SA frio
FDC baixo ($\lvert\rho\rvert \leq 0.15$)	Sem orientação global	GA, DE, PSO (exploração ampla)
FDC negativo (< -0.15)	Paisagem enganosa	SA quente, Tabu com lista longa
ξ pequeno	Rugosidade alta	SA, GA com mutação forte
ξ grande	Paisagem suave	HC, busca local intensa
$\frac{\widehat{M}}{R}$ alto	Alta multimodalidade	GA, DE (população diversa)
ν alto	Platôs extensos	Tabu, SA (aceita movimentos neutros)

Tabela 3: Síntese das métricas de FLA e suas implicações para a escolha de algoritmos. As métricas não são mutuamente exclusivas: um problema pode ter FDC baixo **e** alta neutralidade simultaneamente — como é comum em problemas de roteamento e alocação discreta em EE.

> ⚠️ **Atenção**
> FLA não substitui o teste empírico — mas reduz o espaço de candidatos antes de executar experimentos caros. Em problemas de engenharia onde cada avaliação de $f(\boldsymbol{x})$ envolve uma simulação de fluxo de potência (segundos a minutos por avaliação), escolher o algoritmo errado tem custo real. Um diagnóstico de FLA com $N = 500$ amostras e $R = 100$ reinícios custa uma fração do orçamento de avaliações de um experimento completo.

6 Estudo de Caso: O Problema da Mochila

Antes de avançar para os algoritmos, é útil ancorar todos os conceitos em um problema concreto e intuitivo. O **Problema da Mochila 0-1** (*0-1 Knapsack*) é o ponto de partida ideal: seu enunciado cabe em uma frase, sua formulação matemática conecta diretamente às seções anteriores, e sua aparente simplicidade esconde uma dificuldade combinatória genuína — a mesma que motiva o uso de metaheurísticas em problemas de grande porte.

> 🔁 **Fio Condutor — Problema da Mochila**
> Este problema acompanhará todo o curso. Nas próximas aulas veremos como cada família de metaheurística o ataca de forma distinta: **Hill Climbing** percorrendo o espaço bit a bit; **Simulated Annealing** aceitando piioras temporárias para escapar de soluções medianas; **Algoritmos Genéticos** combinando soluções parcialmente boas por cruzamento; **Busca Tabu** registrando movimentos recentes para forçar exploração. Ao final do curso, você terá um painel comparativo concreto do comportamento de cada método sobre o mesmo problema.

6.1 Descrição do Problema

Uma mochila suporta no máximo W kg. Há n itens disponíveis; cada item i tem um **peso** w_i e um **valor** v_i. O objetivo é selecionar um subconjunto de itens que **maximize o valor total** sem exceder a capacidade da mochila.

A decisão sobre cada item é binária: ou ele vai na mochila ($x_i = 1$) ou não ($x_i = 0$). Não é possível levar frações — isso distingue o problema da versão contínua (fracionária), que é trivialmente resolvida por uma regra gulosa de razão valor/peso.

Item	1	2	3	4	5	6
Peso (kg)	2	3	4	5	1	6
Valor (R\$)	6	7	5	9	3	10
v/w	3,0	2,3	1,3	1,8	3,0	1,7

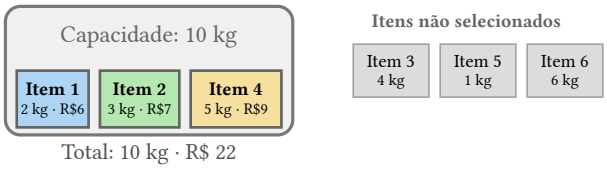

Figura 2: Instância do Problema da Mochila com 6 itens e capacidade $W = 10$ kg. **Acima:** dados de cada item (peso, valor, razão valor/peso). **Abaixo à esquerda:** solução {1, 2, 4} com valor total R$ 22 e peso exato 10 kg. **Abaixo à direita:** itens excluídos. Esta solução é ótima — mas um algoritmo guloso baseado na razão $\frac{v_i}{w_i}$ chegaria a uma conclusão diferente (ver Seção 6.4).

6.2 Formulação Matemática

Para cada item $i \in \{1, ..., n\}$, definimos a variável de decisão binária:

$$x_i \in \{0, 1\}, \quad x_i = 1 \text{ se o item } i \text{ é selecionado, } 0 \text{ caso contrário.}$$

Função objetivo. Maximizar o valor total dos itens selecionados:

$$\max_{\boldsymbol{x}} \quad f(\boldsymbol{x}) = \sum_{i=1}^{n} v_i x_i.$$

Restrição de capacidade. O peso total não pode exceder a capacidade da mochila:

$$\sum_{i=1}^{n} w_i x_i \leq W. \qquad \text{(C1)}$$

Domínio. Cada variável é binária:

$$x_i \in \{0, 1\}, \quad \forall i \in \{1, ..., n\}. \qquad \text{(C2)}$$

A formulação é notavelmente compacta: n variáveis, 1 restrição de desigualdade e 1 restrição de domínio. No entanto, essa aparente simplicidade é enganosa. O espaço de soluções candidatas tem cardinalidade 2^n, e não existe estrutura convexa que permita "descer em direção ao ótimo" a partir de um ponto qualquer.

⚠️ **Atenção**

O Problema da Mochila 0-1 é **NP-difícil**. Não se conhece algoritmo com tempo de execução polinomial no pior caso para instâncias gerais. Para $n = 100$ itens, o espaço tem $2^{\{100\}} \approx 10^{\{30\}}$ soluções — inviável por enumeração. Para $n = 30$, já são mais de 10^9 combinações, tornando a busca exaustiva impraticável mesmo em hardware moderno.

6.3 Classificação do Problema

🔁 **Fio Condutor — Problema da Mochila**

Aplicando os critérios da Tabela 1 ao Problema da Mochila 0-1:

- **Natureza das variáveis:** puramente discreta (inteira binária) — $x_i \in \{0, 1\}$.
- **Linearidade:** o problema é linear nas variáveis (função objetivo e restrição são lineares em $\boldsymbol{x}$); a dificuldade vem inteiramente da integridade binária.
- **Convexidade: não-convexo** — o espaço viável é um conjunto discreto de 2^n pontos; o conceito de convexidade contínua não se aplica diretamente, e a estrutura combinatória cria múltiplos "ótimos locais" sob qualquer vizinhança discreta natural.
- **Número de objetivos:** mono-objetivo na formulação básica; versões bi-objetivo (maximizar valor e minimizar peso simultaneamente) aparecem em planejamento de portfólios e alocação de recursos.
- **Dinâmica:** estático — não há acoplamento temporal entre as decisões.

6.4 A Paisagem de Busca: Visualizando a Dificuldade

Para problemas discretos, a paisagem de busca depende de como definimos **vizinhança**. A escolha mais natural para o Problema da Mochila é a **vizinhança de 1-flip**: dois vetores $\boldsymbol{x}$ e $\boldsymbol{x}'$ são vizinhos se diferem em exatamente um bit.

Com $n = 6$ itens e capacidade $W = 10$ kg, é possível enumerar todas as $2^6 = 64$ soluções candidatas e calcular o valor de cada uma. A Figura 3 mostra essa paisagem completa: cada ponto representa uma solução binária, o eixo vertical é o valor $f(\boldsymbol{x})$ (soluções inviáveis recebem penalidade e aparecem abaixo de zero), e as linhas conectam soluções vizinhas por 1-flip.

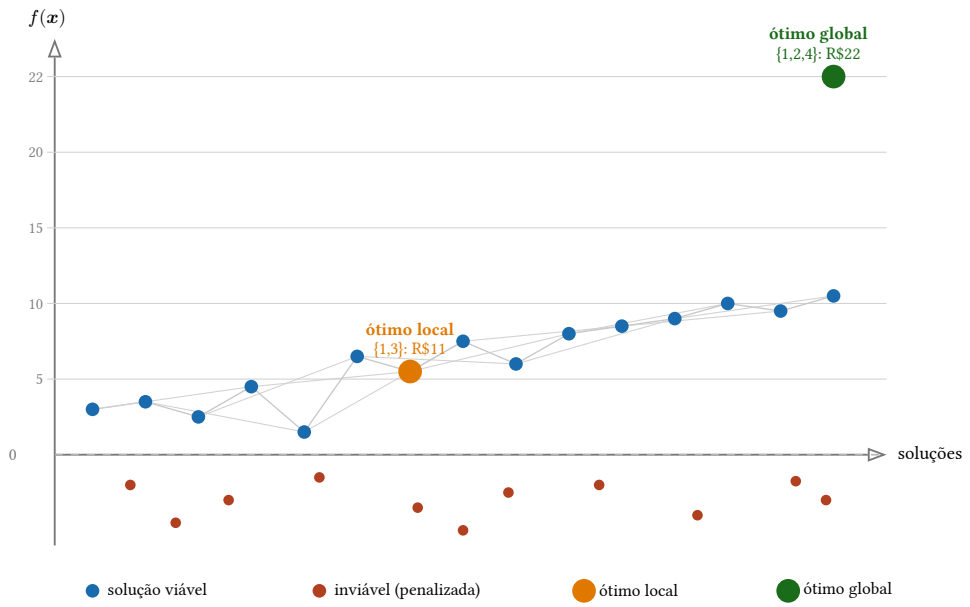

Figura 3: Paisagem de busca esquemática do Problema da Mochila com $n = 6$ itens e $W = 10$ kg. Cada ponto é uma solução binária; linhas conectam soluções vizinhas por 1-flip. Soluções inviáveis aparecem abaixo do eixo (penalizadas). O ótimo local {1, 3} com valor R$ 11 é uma "armadilha": todos os seus vizinhos por 1-flip têm valor menor ou são inviáveis. O ótimo global {1, 2, 4} com valor R$ 22 está em uma bacia separada, inacessível por descida local a partir de {1, 3}.

Fontes de dificuldade combinatória:

- **Ótimos locais sob vizinhança 1-flip.** Uma solução é um ótimo local se nenhuma troca simples (adicionar ou remover um único item) melhora o valor mantendo viabilidade. Esses pontos são "armadilhas" para algoritmos de busca local.
- **Correlação peso-valor.** Itens de alto valor tendem a ser pesados. A tensão entre maximizar valor e respeitar a capacidade cria uma fronteira de Pareto discreta — e muitas soluções de aparência razoável ficam aquém do ótimo por apenas uma troca não-óbvia.
- **Ausência de gradiente.** O espaço é discreto: não existe derivada de f em relação a x_i. A informação de "qual direção melhorar" precisa ser obtida por avaliação explícita de soluções vizinhas — o que tem custo $O(n)$ por iteração.
- **Explosão combinatória.** Para $n = 50$ itens, há $2^{\{50\}} \approx 10^{\{15\}}$ soluções candidatas. Mesmo enumerando 10^9 soluções por segundo, a busca exaustiva levaria mais de 11 dias — e isso **antes** de escalar para instâncias reais.

Exemplo

Armadilha da heurística gulosa. Uma estratégia intuitiva é selecionar itens em ordem decrescente de razão $\frac{v_i}{w_i}$. Para nossa instância:

Item	$\frac{v_i}{w_i}$	Peso acum. (kg)	Ação
1	3,0	2	✓ seleciona
5	3,0	3	✓ seleciona
2	2,3	6	✓ seleciona
4	1,8	11	× excede W

Item	$\frac{v_i}{w_i}$	Peso acum. (kg)	Ação
3	1,3	10	✓ seleciona
6	1,7	—	× excede W

Resultado guloso: {1, 5, 2, 3} com valor R$ 21 e peso 10 kg. A solução é viável e boa — mas **não é ótima**. A solução {1, 2, 4} alcança R$ 22 com o mesmo peso. O problema é que a heurística ficou presa na bacia de atração de sua própria lógica: uma vez que os itens 1, 5 e 2 foram selecionados, nenhuma troca simples (1-flip) revela a melhoria — adicionar o item 4 individualmente viola a capacidade, e remover qualquer item selecionado reduz o valor antes de abrir espaço para a troca vantajosa.

Esse fracasso não é acidental nem corrigível com uma regra gulosa melhor. É uma consequência estrutural da paisagem: **a bacia de atração da solução gulosa não contém o ótimo global**. Qualquer método que siga uma direção de melhoria imediata — sem mecanismo para escapar de vales locais — está sujeito ao mesmo problema, independentemente do critério de ordenação usado.

7 De Heurísticas a Metaheurísticas

O exemplo anterior expõe um limite fundamental. A heurística gulosa é **específica**: a regra de ordenar por $\frac{v_i}{w_i}$ foi construída para o Problema da Mochila e não tem sentido em outros contextos. Ela codifica conhecimento sobre a estrutura particular daquele problema. E mesmo assim falha.

Uma **heurística** é, por definição, um procedimento de busca projetado para um problema específico, explorando suas propriedades estruturais para guiar decisões. Heurísticas bem projetadas são rápidas e frequentemente produzem boas soluções — mas não oferecem garantias, e seu desempenho degrada quando a estrutura do problema viola as suposições embutidas na regra.

A pergunta natural é: existe um nível de abstração acima das heurísticas, capaz de guiar a busca em **qualquer** espaço de soluções, sem depender da estrutura específica do problema?

A resposta é sim — e essa é a ideia central das metaheurísticas.

7.1 Definição e Propriedades Essenciais

Definição — Metaheurística
Uma **metaheurística** é uma estratégia de busca de alto nível, independente do problema, que guia e coordena heurísticas subordinadas para explorar eficientemente o espaço de soluções. Formalmente, opera sobre três componentes abstratos:

- **Representação:** uma codificação $x \in \mathcal{S}$ das soluções candidatas, sem assumir estrutura específica de $\mathcal{S}$.
- **Vizinhança** $\mathcal{N}(x)$: um operador que define quais soluções são "próximas" de x, permitindo movimentos locais.
- **Função objetivo** $f : \mathcal{S} \to \mathbb{R}$: o único canal pelo qual a metaheurística "enxerga" o problema — ela não acessa a estrutura interna de f.

A partir desses componentes, a metaheurística decide **como**, **quando** e **para onde** mover a busca.

Quatro propriedades distinguem metaheurísticas de métodos exatos e de heurísticas simples:

1. Agnóstica ao problema
O mesmo algoritmo (SA, GA, PSO) pode ser aplicado ao Knapsack, ao OPF e ao planejamento de rotas — basta trocar a representação e a função objetivo.

2. Mecanismo anti-estagnação
Toda metaheurística possui alguma estratégia para escapar de ótimos locais: aceitar piores temporárias (SA), manter memória de movimentos recentes (Tabu), combinar soluções distintas (GA).

3. Sem garantia de otimalidade
Metaheurísticas não provam que encontraram o ótimo global. O compromisso é prático: **boas soluções em tempo razoável**, não a solução perfeita.

4. Equilíbrio exploração/explotação
A tensão central de toda metaheurística: **explorar** regiões novas do espaço (diversificação) versus **explotar** as regiões promissoras já encontradas (intensificação).

⚠️ **Atenção**
Não existe consenso na literatura sobre uma definição única de metaheurística — o termo foi cunhado por Glover (1986) para a Busca Tabu e desde então expandiu-se para cobrir famílias muito distintas de algoritmos. O que unifica todas as definições são as propriedades acima: independência do problema, mecanismo de escape de ótimos locais e ausência de garantia de otimalidade global.

7.2 Famílias de Metaheurísticas

As metaheurísticas podem ser organizadas em duas grandes famílias segundo o número de soluções mantidas simultaneamente durante a busca.

Busca com solução única
(Trajectory methods)
Mantém e modifica uma única solução corrente. A busca traça uma trajetória no espaço de soluções.

- **Hill Climbing** — aceita apenas melhoras
- **Simulated Annealing** — aceita piores com probabilidade decrescente
- **Busca Tabu** — usa memória para proibir revisitas

Busca com população
(Population-based methods)
Mantém um conjunto de soluções que evoluem coletivamente. A diversidade da população substitui os mecanismos explícitos de escape.

- **Algoritmos Genéticos** — seleção, cruzamento, mutação
- **Evolução Diferencial** — perturbação vetorial
- **PSO / ACO** — inteligência de enxame

A Tabela 4 situa cada método no mapa do curso e resume o mecanismo de escape de cada família.

Método	Família	Mecanismo de escape	Aula
Hill Climbing	Solução única	Reinício aleatório	3
Simulated Annealing	Solução única	Aceita pioras ($e^{\{-\frac{\Delta}{T}\}}$)	3
Busca Tabu	Solução única	Memória de movimentos proibidos	4
Algoritmos Genéticos	População	Cruzamento entre soluções	5
Evolução Diferencial	População	Perturbação vetorial	5
PSO	Enxame	Atração social e cognitiva	6
ACO	Enxame	Feromônio e esquecimento	6

Tabela 4: Famílias de metaheurísticas estudadas no curso e seus mecanismos de escape de ótimos locais.

Com esse mapa em mãos, as próximas aulas podem ser lidas com propósito: cada algoritmo é uma resposta diferente à mesma pergunta — **como escapar de armadilhas num espaço de busca que não podemos ver por inteiro?**

8 Exercícios

Exercício

1 (Convexidade) - *1,0 ponto.* Seja $f : \mathbb{R} \to \mathbb{R}$ uma função convexa. Usando apenas a definição da Seção 3.2, demonstre que f não pode possuir dois mínimos locais distintos com valores diferentes — isto é, se $\boldsymbol{x}^*$ é mínimo global de f, então todo mínimo local de f tem o mesmo valor $f(\boldsymbol{x}^*)$.

Sugestão: suponha por contradição que existem dois mínimos locais $\boldsymbol{x}_1^l$ e $\boldsymbol{x}_2^l$ com $f(\boldsymbol{x}_1^l) < f(\boldsymbol{x}_2^l)$, construa o ponto convexo $\boldsymbol{z} = \lambda \boldsymbol{x}_1^l + (1-\lambda)\boldsymbol{x}_2^l$ para λ próximo de 1, e mostre que $f(\boldsymbol{z}) < f(\boldsymbol{x}_2^l)$, contradizendo a minimalidade local de $\boldsymbol{x}_2^l$.

Exercício

2 (Análise Experimental) - *1,0 ponto.* Implemente em Python o seguinte experimento comparativo sobre o Problema da Mochila com $n = 20$ itens, pesos $w_i \in \text{Uniforme}(1, 15)$ e valores $v_i \in \text{Uniforme}(1, 20)$ e capacidade $W = \frac{1}{2}\sum w_i$:

(a) Geração da instância. Fixe uma semente global (`numpy.random.seed(42)`) e gere os vetores $\boldsymbol{w}$ e $\boldsymbol{v}$. Imprima a instância gerada de forma legível.

(b) Busca aleatória pura. Execute 30 repetições independentes (sementes 0 a 29). Em cada repetição, amostre uniformemente 10^4 soluções binárias e registre o maior valor viável encontrado. Reporte média e desvio padrão dos 30 resultados.

(c) Heurística gulosa. Ordene os itens por razão $\frac{v_i}{w_i}$ em ordem decrescente e selecione itens enquanto a capacidade não for excedida. Como a heurística é determinística, execute-a uma única vez e registre o valor obtido.

(d) Comparação. Apresente os resultados em uma tabela com três colunas — Método, Valor médio (ou único), Desvio padrão — e responda: em quantas das 30 execuções aleatórias o valor encontrado superou o resultado guloso? O que isso sugere sobre a eficácia relativa de cada abordagem?

> *Observação:* o objetivo deste exercício não é obter a solução ótima, mas desenvolver intuição sobre o comportamento de métodos simples antes de estudar metaheurísticas.

Leituras recomendadas:

- Nocedal, J., Wright, S. J. **Numerical Optimization**. 2ª ed. Springer, 2006. Caps. 1 e 12.
- Talbi, E.-G. **Metaheuristics: From Design to Implementation**. Wiley, 2009. Caps. 1–2.
- Kellerer, H., Pferschy, U., Pisinger, D. **Knapsack Problems**. Springer, 2004. Cap. 1.
- Glover, F., Kochenberger, G. (eds.) **Handbook of Metaheuristics**. Springer, 2003. Cap. 1.